

LAB3 Report

Pin-Jing, Li (111511015 *ouo.ee11@nycu.edu.tw*)
Ching-Kai, Huang (111511151 *kevin.ee11@nycu.edu.tw*)
Duan-Kai, Hu (112511015 *huduankai@gmail.com*)

November 9, 2025

1 Data Fetching and Converting

The first thing is to extract the data from the raw txt output with messy logs and timestamps. We first reads an nRF Connect text log and extracts every hex byte string that matches "(0x) ...". For each "frame," we searches the first framing marker FF 0F and treats the next 12 bytes as the payload.

```
1 % ----- Input / output file names -----
2 inFile = 'car03_202510221410.txt';
3 outFile = 'car03.csv';
4
5 % ----- Step 1: Extract byte strings from log -----
6 txt = fileread(inFile);
7
8 % Matches lines like: (0x)
9 pattern = '\(0x\)s*([0-9A-Fa-f]([0-9A-Fa-f](?:-[0-9A-Fa-f]([0-9A-Fa-f]))*))';
10 tokens = regexp(txt, pattern, 'tokens');
11 frames = cellfun(@(x) x{1}, tokens, 'UniformOutput', false);
12
13 data = []; % [N x 6]
14 n_skipped = 0;
15
16 for i = 1:numel(frames)
17     % Convert "AA-BB-..." -> uint8 vector (force ROW vector)
18     hexParts = split(frames{i}, '-'); % nx1 string array
19     raw = uint8(hex2dec(hexParts)).'; % 1 x M (transpose to row)
20
21     % Basic length guard
22     if numel(raw) < 14 % minimal size to even hold FF-0F + 12 bytes
23         n_skipped = n_skipped + 1;
24         continue;
25     end
26
27     % Find first occurrence of the framing marker 0xFF 0x0F
28     seq = double(raw); % ensure it's a row vector for strfind
29     k = strfind(seq, [255 15]); % positions where FF 0F begins
30
31     % Optional alternate ordering check (rare, but keep it robust)
32     alt_used = false;
33     if isempty(k)
34         k = strfind(seq, [15 255]);
35         if isempty(k)
36             n_skipped = n_skipped + 1;
37             continue;
38         else
39             alt_used = true;
40         end
41     end
42
43     payload_start = k(1) + 2; % first byte AFTER the 2-byte marker
44
```

```

45 % We need 12 payload bytes for 6 int16 values
46 if payload_start + 12 - 1 > numel(raw)
47     n_skipped = n_skipped + 1;
48     continue;
49 end
50
51 p = raw(payload_start : payload_start + 12 - 1); % 1x12 payload
52
53 % Little-endian int16 decode (low byte first, then high byte)
54 acc_x = typecast(uint16(p(1)) + bitshift(uint16(p(2)), 8), 'int16');
55 acc_y = typecast(uint16(p(3)) + bitshift(uint16(p(4)), 8), 'int16');
56 acc_z = typecast(uint16(p(5)) + bitshift(uint16(p(6)), 8), 'int16');
57 gyro_x = typecast(uint16(p(7)) + bitshift(uint16(p(8)), 8), 'int16');
58 gyro_y = typecast(uint16(p(9)) + bitshift(uint16(p(10)), 8), 'int16');
59 gyro_z = typecast(uint16(p(11)) + bitshift(uint16(p(12)), 8), 'int16');
60
61 data = [data; double([acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z])]; %#ok<AGROW>
62 end
63
64 % ----- Step 3: Write results to CSV -----
65 headers = {'acc_x', 'acc_y', 'acc_z', 'gyro_x', 'gyro_y', 'gyro_z'};
66 T = array2table(data, 'VariableNames', headers);
67 writetable(T, outFile);
68
69 % ----- Step 4: Plot results -----
70 figure('Name', 'Accelerometer and Gyroscope Data', 'NumberTitle', 'off');
71
72 subplot(2,1,1);
73 plot(data(:,1:3), 'LineWidth', 1);
74 title('Accelerometer');
75 xlabel('Sample Index');
76 ylabel('Raw Value');
77 legend('acc_x', 'acc_y', 'acc_z', 'Location', 'best');
78 grid on;
79
80 subplot(2,1,2);
81 plot(data(:,4:6), 'LineWidth', 1);
82 title('Gyroscope');
83 xlabel('Sample Index');
84 ylabel('Raw Value');
85 legend('gyro_x', 'gyro_y', 'gyro_z', 'Location', 'best');
86 grid on;

```

2 Experiment I: Car movement

2.1 Calibration

This section talks about the calibration on the acceleration detector of the IMU. We make use of the gravity $1g = 16384$ as our reference. The work flow is as follow,

- 1 Face the target direction upward and collect the signal.
- 2 Obtain the mean of the target direction in raw unit with `meas_mean = mean(target_axis)`
- 3 Obtain the desired correction term `b_axis = a_exp - meas_mean` with `a_exp = 16384`

The following set of figures shows the signal before and after the calibration.

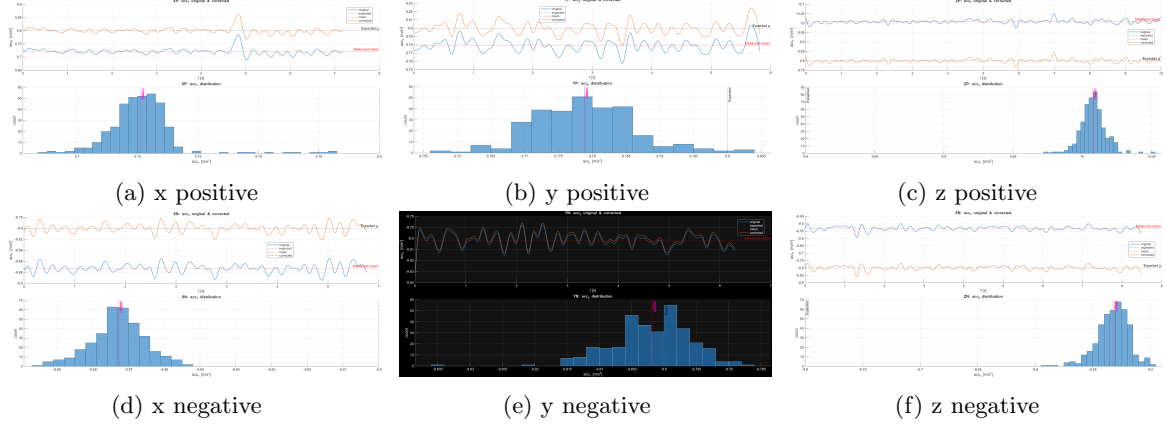


Figure 1: Calibration of acceleration on six different direction

2.2 Car Displacement

Data Preprocessing In this preprocessing stage, the raw IMU data matrix $X \in \mathbb{R}^{N \times 6}$, consisting of the tri-axial accelerometer and gyroscope signals, is first converted to double precision and time-stamped by $t = (0 : N - 1) / F_s$.

A detrending operation is then applied to remove bias or slow drifts according to the calibration we've done in the previous stage.

We then pass our signal through a low-pass Butterworth filter of order `ord` and cutoff frequency f_c is designed and applied, via zero-phase filtering (`filtfilt`), exclusively to the accelerometer channels, thereby suppressing high-frequency noise while preserving temporal alignment.

```

1 X = double([T{:,ix_accx}, T{:,ix_accy}, T{:,ix_accz}, ...
2           T{:,ix_gyrx}, T{:,ix_gyry}, T{:,ix_gyryz}]);
3 [N,C] = size(X);
4 t = (0:N-1)'/Fs;
5
6 %% ----- Preprocess -----
7 X_pre = X;
8 X_pre = detrend(X_pre, 0);
9
10 [b,a] = butter(ord, fc/(Fs/2), 'low');
11 X_pre = filtfilt(b,a,X_pre);

```

Drift Correction We design an automatic motion segmentation before integrating the total displacement.

We first form a scalar detection signal from the preprocessed accelerometer per-axis magnitude $|a_{a_i}(t)|$, then apply a simple amplitude test $\text{active}(t) \leftarrow (\text{sig_for_detect}(t) > \text{acc_threshold})$ and extract contiguous runs with `logical_runs`, yielding raw time windows `win_raw` = $[t_{\text{start}}, t_{\text{stop}}]$.

We then prune spuriously short bursts by enforcing a minimum duration `min_burst_duration`, coalesce adjacent bursts separated by gaps shorter than `min_silent_gap` via `merge_gaps`, and finally dilate each window by `dilate_edges` seconds on both sides (clipped to $[0, t_{\text{max}}]$) to capture boundary transients. The result `win` consists of robust, edge-padded motion intervals suitable for subsequent velocity/displacement integration and gait-phase analysis.

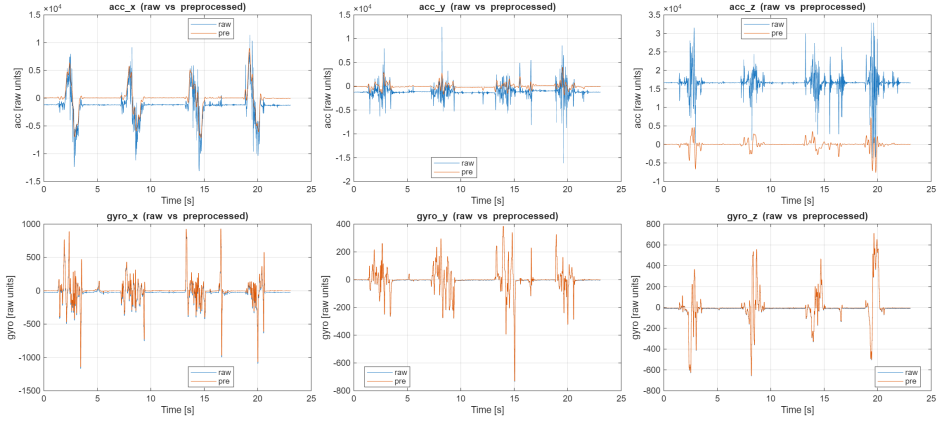


Figure 2: Acc and Gyr signal before and after the preprocessing, with calibration and filtering.

```

1 %% ----- Auto-detect motion windows -----
2 dt = 1/Fs;
3 ai = pick_axis(axis_for_integration);
4 acc_axis = X_pre(:,ai) * g_to_mps2;
5 am = sqrt(sum((X_pre(:,1:3)*g_to_mps2).^2,2));
6 sig_for_detect = am;
7
8 active = sig_for_detect > acc_threshold;
9 [starts, stops] = logical_runs(active);
10
11 % To times
12 win_raw = [t(starts), t(stops)];
13
14 % Keep only sufficiently long bursts
15 dur = win_raw(:,2) - win_raw(:,1);
16 win = win_raw(dur >= min_burst_duration, :);
17
18 % Merge short gaps
19 win = merge_gaps(win, min_silent_gap);
20
21 % Dilate edges
22 win = [max(0, win(:,1)-dilate_edges), min(t(end), win(:,2)+dilate_edges)];

```

After the motion detection windows was created, we perform drift correction on each windows based on the equation in TA's instruction.

$$v_d(t) = \frac{v_e}{t_{se} - t_{ss}} \times (t - t_{ss}) \quad (1)$$

$$v_c(t) = v_r(t) - v_d(t) \quad (2)$$

The drift-correction is as shown in the figure below.

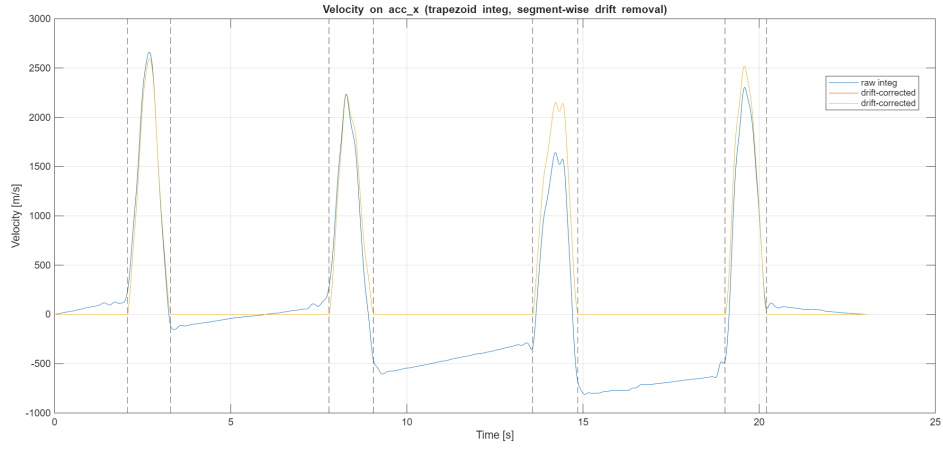


Figure 3: X-axis velocity by integrating the pre-processed acceleration, before and after drift correction.

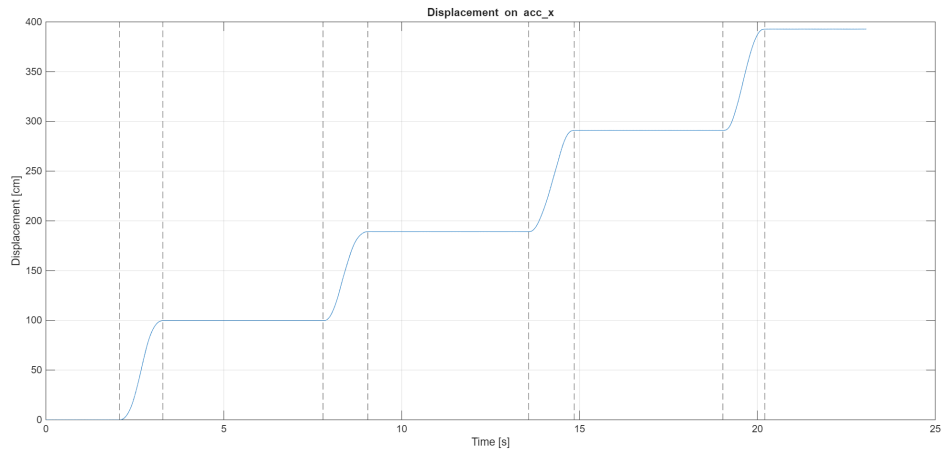


Figure 4: Car displacement by integrating over the drift-corrected velocity.

3 Experiment II: Walking Motion Detection

3.1 Calibration and Preprocessing

Since what we're interested in is the total displacement on the three direction, in this section we directly pass the signal through a low pass filter.

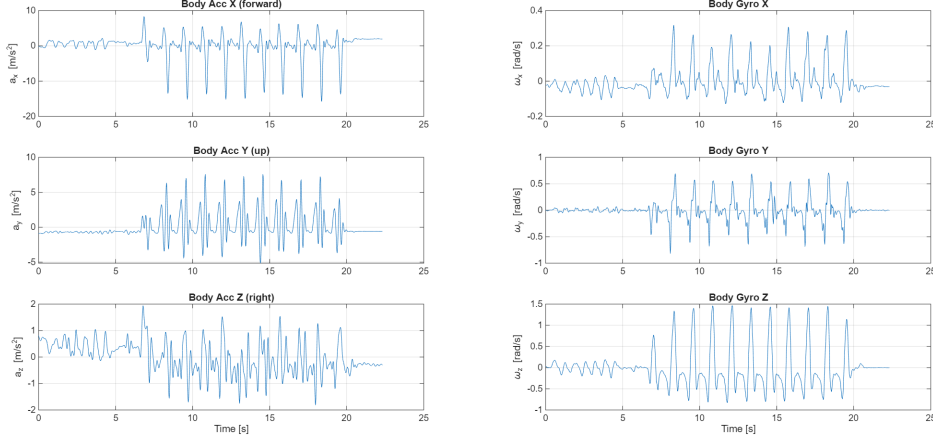


Figure 5: Pre-processed Body-frame data

3.2 Mahony Filter for IMU Orientation Estimation

Let us talk about Quaternions. I'm so happy you mentioned about this since I've been recently obsessed with William R. Hamilton.

The Mahony filter [1] is a computationally efficient attitude estimation algorithm designed to fuse high-rate, drift-prone gyroscope data with low-rate, drift-free accelerometer (and optionally magnetometer) measurements. Its goal is to produce a stable quaternion-based representation of the sensor's orientation relative to the earth frame, denoted as the body-to-earth rotation.

3.2.1 Quaternion Kinematics

The time evolution of orientation is governed by the quaternion differential equation

$$\dot{q} = \frac{1}{2} q \otimes \omega_q, \quad (3)$$

where $q = [q_0, q_1, q_2, q_3]^\top$ represents the unit quaternion, and $\omega_q = [0, \boldsymbol{\omega}]^\top$ encodes the angular velocity vector measured by the gyroscope. Direct integration of this equation yields a predicted orientation; however, gyroscope drift causes cumulative errors that must be corrected through feedback.

3.2.2 Feedback Correction Law

Let $\mathbf{g}_b$ denote the gravity vector measured by the accelerometer in the body frame, and let $\hat{\mathbf{g}}_b$ denote the expected gravity vector computed from the current quaternion estimate. The orientation error is then defined as

$$\mathbf{e} = \mathbf{g}_b \times \hat{\mathbf{g}}_b. \quad (4)$$

A proportional–integral (PI) controller applies this error as a feedback term to correct the gyroscope measurements:

$$\boldsymbol{\omega}_{\text{corr}} = \boldsymbol{\omega} + k_p \mathbf{e} + k_i \int \mathbf{e} dt, \quad (5)$$

where k_p and k_i are proportional and integral gains, respectively. The corrected angular rate is then used to update the quaternion:

$$\dot{q} = \frac{1}{2} q \otimes [0, \boldsymbol{\omega}_{\text{corr}}]^\top. \quad (6)$$

The quaternion is normalized after each integration step to ensure $\|q\| = 1$.

3.2.3 Rotation Matrix Conversion

The corresponding rotation matrix from body to earth frame, R_{be} , is derived from the quaternion as

$$R_{be} = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}. \quad (7)$$

This matrix transforms any body-frame vector $\mathbf{v}_b$ into the earth frame via

$$\mathbf{v}_e = R_{be} \mathbf{v}_b. \quad (8)$$

3.2.4 Characteristics

The Mahony filter offers a balance between accuracy and computational simplicity:

- **Lightweight computation:** Uses only vector algebra and quaternion arithmetic—no matrix inversions are required.
- **Low latency:** Well-suited for real-time embedded systems.
- **Robust to noise:** The proportional-integral feedback attenuates both high-frequency sensor noise and low-frequency drift.
- **Complementary fusion:** Effectively blends high-frequency gyroscope information with low-frequency accelerometer feedback.

In practice, this method provides a stable and drift-compensated estimate of sensor orientation, enabling accurate gravity removal, velocity integration, and gait-phase analysis in IMU-based motion processing systems.

3.2.5 Implementation

The following demonstrate a Matlab implementation of this Conversion.

```

1 %% ----- Orientation -----
2 dt = 1/Fs;
3 q = [1;0;0;0]; % quaternion w,x,y,z (body->earth, earth Z-up)
4 bias = [0;0;0];
5
6 % helpers
7 qmul = @(q1,q2)[ q1(1)*q2(1)-dot(q1(2:4),q2(2:4));
8                 q1(1)*q2(2:4)+q2(1)*q1(2:4)+cross(q1(2:4),q2(2:4)) ];
9 q2R = @(q)[ 1-2*(q(3)^2+q(4)^2), 2*(q(2)*q(3)-q(1)*q(4)), 2*(q(2)*q(4)+q(1)*q(3));
10            2*(q(2)*q(3)+q(1)*q(4)), 1-2*(q(2)^2+q(4)^2), 2*(q(3)*q(4)-q(1)*q(2));
11            2*(q(2)*q(4)-q(1)*q(3)), 2*(q(3)*q(4)+q(1)*q(2)), 1-2*(q(2)^2+q(3)^2) ];
12 normalize = @(v) v./max(norm(v),1e-12);
13
14 Q = zeros(4,N); Rbe_all = zeros(3,3,N);
15
16 n0 = min(N, round(2*Fs));
17
18 acc0 = mean(accB_f(1:n0,:)).'; % 3x1
19 acc0 = acc0 ./ max(norm(acc0), 1e-12); % unit
20 g_e = [0;0;-1]; % 3x1, earth Z-up (gravity direction)
21
22 % rotate acc0 to -g_e
23 v = cross(acc0, -g_e); % 3x1
24 s = norm(v);
25 c = dot(acc0, -g_e);
26 if s < 1e-8
27     q = [1;0;0;0];
28 else
29     axis = v / s; % 3x1
30     angle = atan2(s, c);
31     q = [cos(angle/2); axis * sin(angle/2)]; % 4x1: [w;x;y;z]

```

```

32 end
33
34
35 for k = 1:N
36     % prediction with gyro (body frame)
37     omega_b = (gyrB_f(k,:)') - bias; % rad/s
38     dq = [1; 0.5*omega_b*dt]; % infinitesimal
39     q = normalize(qmul(q, dq));
40
41     % correction: keep gravity vertical
42     Rbe = q2R(q);
43     a_meas = accB_f(k,:)'; a_norm = norm(a_meas);
44     use_acc = (a_norm > 0.8*g0) && (a_norm < 1.2*g0);
45     if use_acc
46         a_b_meas_u = a_meas / a_norm;
47         g_b_est = Rbe.' * g_e; % expected gravity dir (body)
48         e_tilt = cross(a_b_meas_u, g_b_est); % tilt error
49     else
50         e_tilt = [0;0;0];
51     end
52     omega_corr = Kp * e_tilt;
53     bias = bias - Ki * e_tilt * dt;
54
55     % corrected re-integration
56     dq = [1; 0.5*(omega_b + omega_corr)*dt];
57     q = normalize(qmul(q, dq));
58
59     Q(:,k) = q;
60     Rbe_all(:,k) = q2R(q);
61 end
62
63 %% ----- Rotate to earth & remove gravity
64 accE = zeros(N,3); gyrE = zeros(N,3);
65 for k = 1:N
66     Rbe = Rbe_all(:,k);
67     gyrE(k,:) = (Rbe * gyrB_f(k,:).')'; % rad/s in earth
68     accE(k,:) = (Rbe * accB_f(k,:).')'; % m/s^2 in earth
69 end
70 accE = accE - [0 0 g0]; % subtract gravity

```

The resulted coordinate-transformed signals are plotted as below.

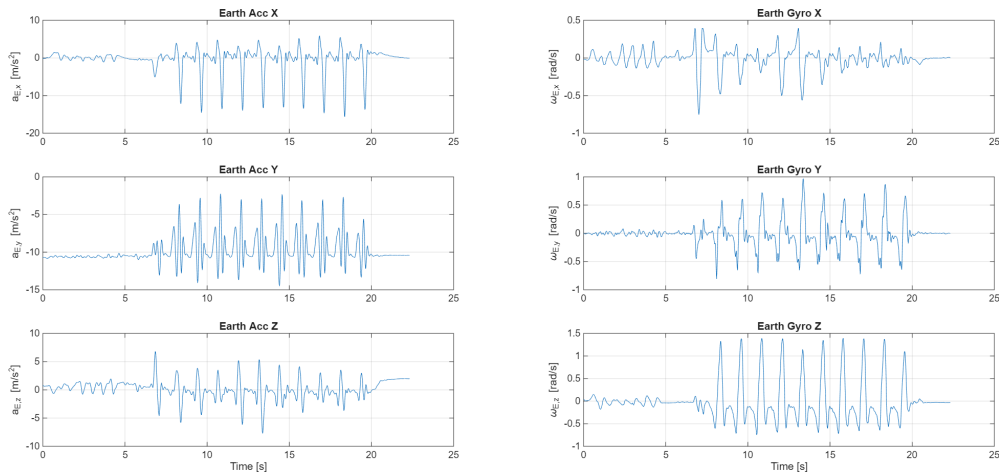


Figure 6: Earth-frame data

3.3 Walking Motion Analysis

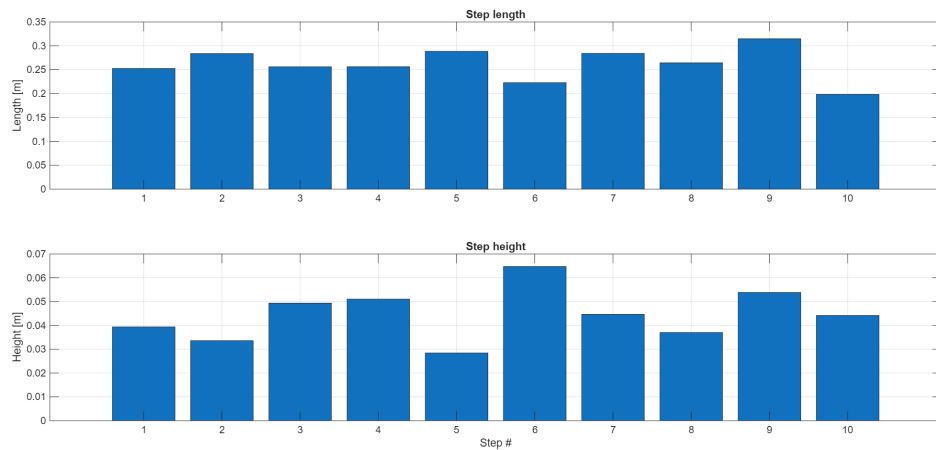


Figure 7

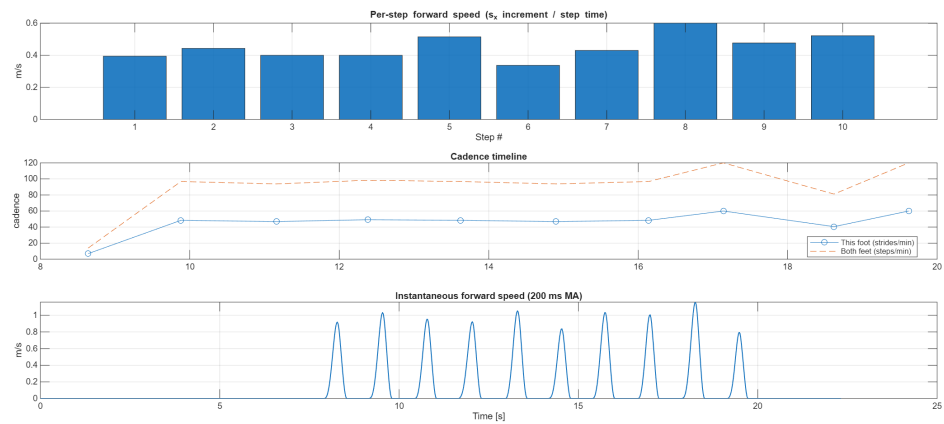
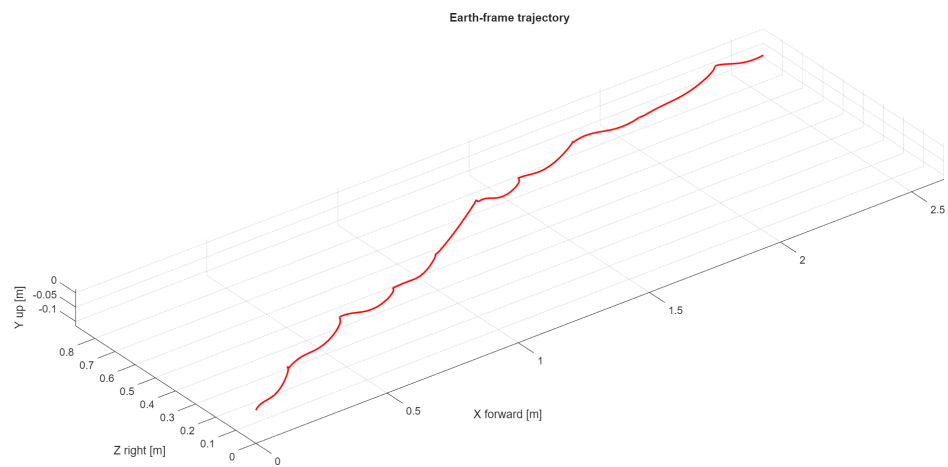


Figure 8

3.4 Walking Trace Reconstruction



I'm dead in my midterm so I'm not even bothered by the square motion.

References

- [1] R. Mahony, T. Hamel, and J.-M. Pflimlin, “Nonlinear Complementary Filters on the Special Orthogonal Group,” *IEEE Transactions on Automatic Control*, vol. 53, no. 5, pp. 1203–1218, 2008. doi:[10.1109/TAC.2008.923738](https://doi.org/10.1109/TAC.2008.923738).